Task 5.3 Report

1-Adversarial Attacks:

Fast Gradient Sign Method: an adversarial attack type expose the vulnerability of modern models to minor variations in input data. These perturbations, which frequently go unnoticed by human observers, inflict errors on prediction accuracy.

```
def fgsm_attack(model, images, labels, epsilon,
                device,
                mean=(0.5,0.5,0.5), std=(0.5,0.5,0.5),
                loss_fn=torch.nn.CrossEntropyLoss()):
    """
```

Create adversarial examples on `images` using single-step FGSM.

“””

Args:

model: PyTorch model (should be on device)

images: input batch tensor, normalized already (batch, C, H, W)

labels: true labels (or target labels for targeted attack)

epsilon: perturbation size in *pixel space* (0..1). e.g. 0.01, 0.03.

device: cuda or cpu device

mean,std: normalization used for the inputs (tuples/lists of length C)

loss_fn: loss used to craft the attack (usually CrossEntropy for untargeted)

Returns:

perturbed_images: adversarial batch tensor (same shape as images)

“””

```
    # Ensure copies and device
    images = images.clone().detach().to(device)
    labels = labels.to(device)
    model.eval()
```

Cloning image to avoid affecting original images , then applying device to images and labels which in our case will be cpu.
The reason model is set to evaluation mode is in train() mode Dropout randomly zeroes activations on each forward pass. That randomness would change the gradient w.r.t. the input each time, making the attack noisy and unstable. eval() turns Dropout off.

```
# make inputs require gradients
images.requires_grad = True

# forward
outputs = model(images)
loss = loss_fn(outputs, labels)

# backward on input
model.zero_grad()
loss.backward()

# gradient of loss w.r.t. normalized images
grad = images.grad.data  # shape (N,C,H,W)
```

The model will output predictions for the input images and then compute loss . Instead of trying to minimize loss as usual , the function perform the exact opposite.

```
std_t = torch.tensor(std, device=device).view(1, -1, 1, 1)
eps_norm = epsilon / std_t  # broadcastable to (N,C,H,W)
```

Epsilon have to be converted from pixel space to normalized space to match input

 std_t = torch.tensor(std, device=device).view(1, -1, 1, 1) reshapes the per-channel std into shape (1, C, 1, 1) so eps_norm broadcasts correctly to (N, C, H, W) and applies a separate scaling for each channel.

```
    # FGSM perturbation in normalized space (untargeted): add sign(grad)
    perturbed = images + eps_norm * grad.sign()
```

The most important line in the function it nudges each pixel a small step in the direction that most increases loss, scaled per-channel to respect the epsilon budget.

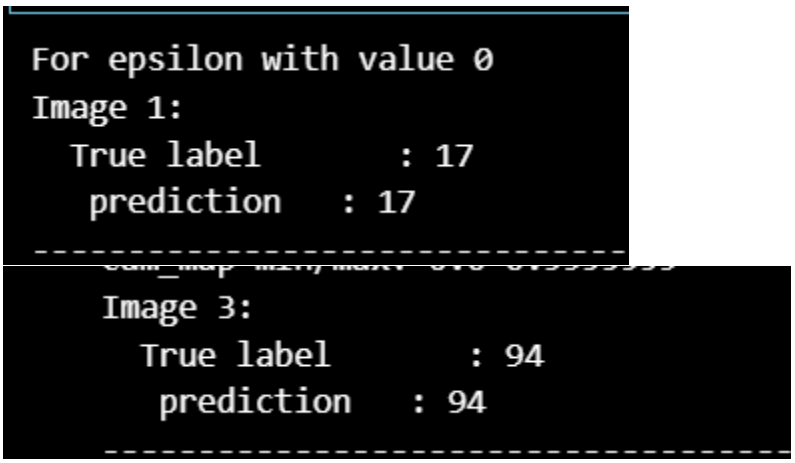
```
    # clamp to valid normalized range so pixel values remain in [0,1]
    mean_t = torch.tensor(mean, device=device).view(1, -1, 1, 1)
    min_norm = (0.0 - mean_t) / std_t
    max_norm = (1.0 - mean_t) / std_t
    perturbed = torch.max(torch.min(perturbed, max_norm), min_norm)

    return perturbed.detach()
```

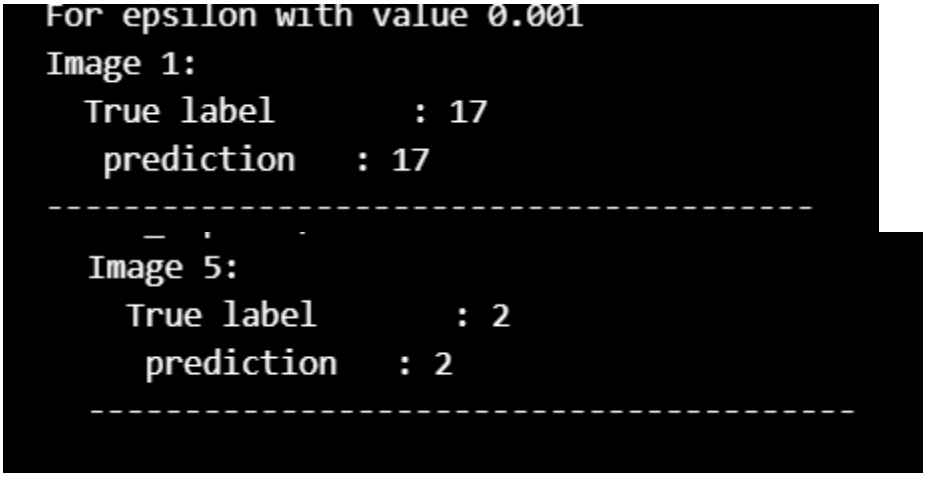
After adding the FGSM perturbation in **normalized space**, the tensor perturbed might contain values that, when converted back to pixel space, would fall outside the valid image range [0,1].

These lines **clamp** the normalized values so that, after de-normalizing, every pixel stays within [0,1]. That preserves valid images and ensures the perturbation remains realistic.

I tested the model performance on different values of epsilons for multiple samples , I will show 2 results for each value
At epsilon = 0 model predicted all results correctly



Epsilon=0.001 explainable techniques showed that model was alittle bit distracted but still managed to predict all samples correctly



Epsilon=0.01 model couldn’t predict nearly half of the samples right

```
orig min/max: 0.0 1.0
cam_map min/max: 0.0 0.9999999
Image 4:
  True label       : 33
   prediction   : 5
---------------------------------------
Image 3:
  True label       : 94
   prediction   : 94
---------------------------------------
```

Epsilon=0.1 model performance started to degrade

```
orig min/max: 0.0 1.0
cam_map min/max: 0.0 0.9999999
Image 3:
  True label       : 94
   prediction   : 100
---------------------------------------
Image 4:
  True label       : 33
   prediction   : 6
---------------------------------------
```

Epsilon =0.2 at this points all samples were misclassified

```
For epsilon with value 0.2
Image 1:
  True label       : 17
   prediction   : 63
---------------------------------------

 cam_map min/max: 0.0 0.9999999
 Image 2:
   True label       : 94
    prediction   : 63
 ------------------------------------
```

2-Defense Mechanisms

Adversarial Training: That is like the normal training of model but difference is you insert attacked samples to train the model on. There is some variations to you can change dataset so that some images are noisy or during training apply an attack mechanism on some inputs.

```
for i, (inputs, labels) in enumerate(train_loader):
    inputs = inputs.to(device)
    labels = labels.to(device)

    batch_size = inputs.size(0)
    perm = torch.randperm(batch_size, device=device)
    split = batch_size // 2
    idx_adv = perm[:split]          # will be attacked
    idx_clean = perm[split:]        # will remain clean

    inputs_adv = inputs[idx_adv] if idx_adv.numel() > 0 else None
    labels_adv = labels[idx_adv] if idx_adv.numel() > 0 else None
    inputs_clean = inputs[idx_clean] if idx_clean.numel() > 0 else None
    labels_clean = labels[idx_clean] if idx_clean.numel() > 0 else None
```

That's the technique I used for every batch I divided into half to be passed into the attacking function and other half be processed normally

```
    # Zero gradients and do a single forward/backward on mixed data
    optimizer.zero_grad()

    loss = 0.0
    if inputs_clean is not None and inputs_clean.size(0) > 0:
        outputs_clean = model_copy(inputs_clean)
        loss_clean = criterion(outputs_clean, labels_clean)
        loss += (inputs_clean.size(0) / batch_size) * loss_clean
    else:
        outputs_clean = None

    if adv_inp_half is not None and adv_inp_half.size(0) > 0:
        outputs_adv = model_copy(adv_inp_half)
        loss_adv = criterion(outputs_adv, labels_adv)
        loss += (adv_inp_half.size(0) / batch_size) * loss_adv
    else:
        outputs_adv = None

    # Backprop once and step
    loss.backward()
    optimizer.step()
```

Output predictions for each , compute loss and perform backward propagation , so tried to obtain a mix in my dataset and train model on it

```
            batch_loss += (inputs_clean.size(0) / batch_size) * loss_clean
            _, preds_clean = torch.max(outputs_clean, 1)
            val_correct += torch.sum(preds_clean == labels_clean).item()
        else:
            outputs_clean = None

        if adv_inp_half is not None and adv_inp_half.size(0) > 0:
            outputs_adv = model_copy(adv_inp_half)
            loss_adv = criterion(outputs_adv, labels_adv)
            batch_loss += (adv_inp_half.size(0) / batch_size) * loss_adv
            _, preds_adv = torch.max(outputs_adv, 1)
            val_correct += torch.sum(preds_adv == labels_adv).item()
        else:
            outputs_adv = None

    val_loss += batch_loss.item() * batch_size

# average validation loss and accuracy over val set
val_loss = val_loss / len(val_subset)
val_acc = val_correct / len(val_subset)

print(f"Epoch [{epoch + 1}/{num_epochs}] train Loss: {train_loss:.4f} val Loss: {val_loss:.4f} val Acc: {val_acc:.4f}")
```

Similarily in validaton but also calculated accuracy , to be considered that before I started training , I splitted the training dataset to train and validation

```
Epoch [1/10] train Loss: 1.3217 val Loss: 1.2033 val Acc: 0.7135
Epoch [2/10] train Loss: 1.3214 val Loss: 1.2345 val Acc: 0.6991
Epoch [3/10] train Loss: 1.2926 val Loss: 1.3145 val Acc: 0.6955
Epoch [4/10] train Loss: 1.3269 val Loss: 1.2899 val Acc: 0.6940
Epoch [5/10] train Loss: 1.3013 val Loss: 1.2466 val Acc: 0.7041
Epoch [6/10] train Loss: 1.3196 val Loss: 1.3230 val Acc: 0.6904
```

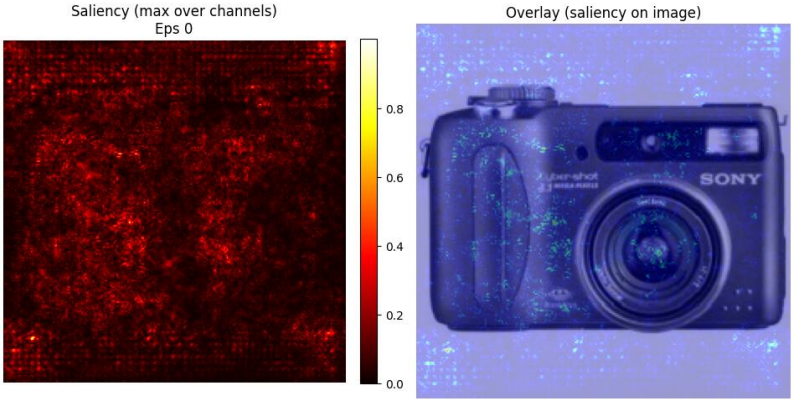
Unfortunately results were below expected maybe the technique of mixing was wrong , epsilon was too high or I should have used another attacking mechanism

3-Model Explainabilty analysis

1)Saliency maps
This maps explain the pixels the model focused on to generate logits. This gradient highlights the pixels that, if changed slightly, would have the largest impact on the model's output.

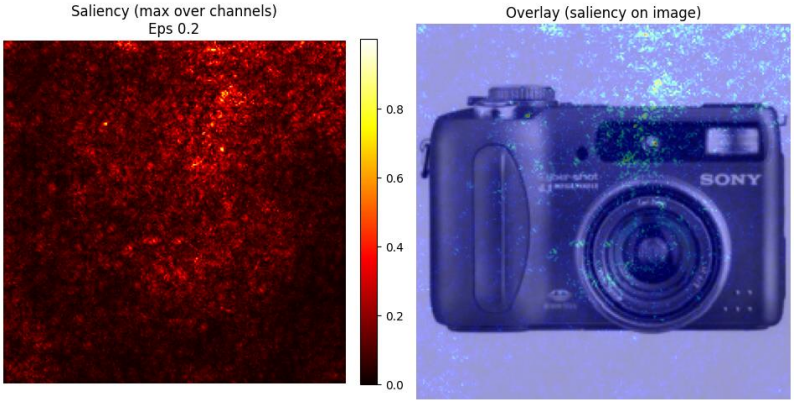
2)GradMap
Produces a heat map to highlight the crucial image regions that contribute to a model's classification decision.
The main difference from saliency maps is It works by using the gradients of the network's output with respect to the feature maps in the final convolutional layer


0
25
50
75
100
125
150
175
200
SONY

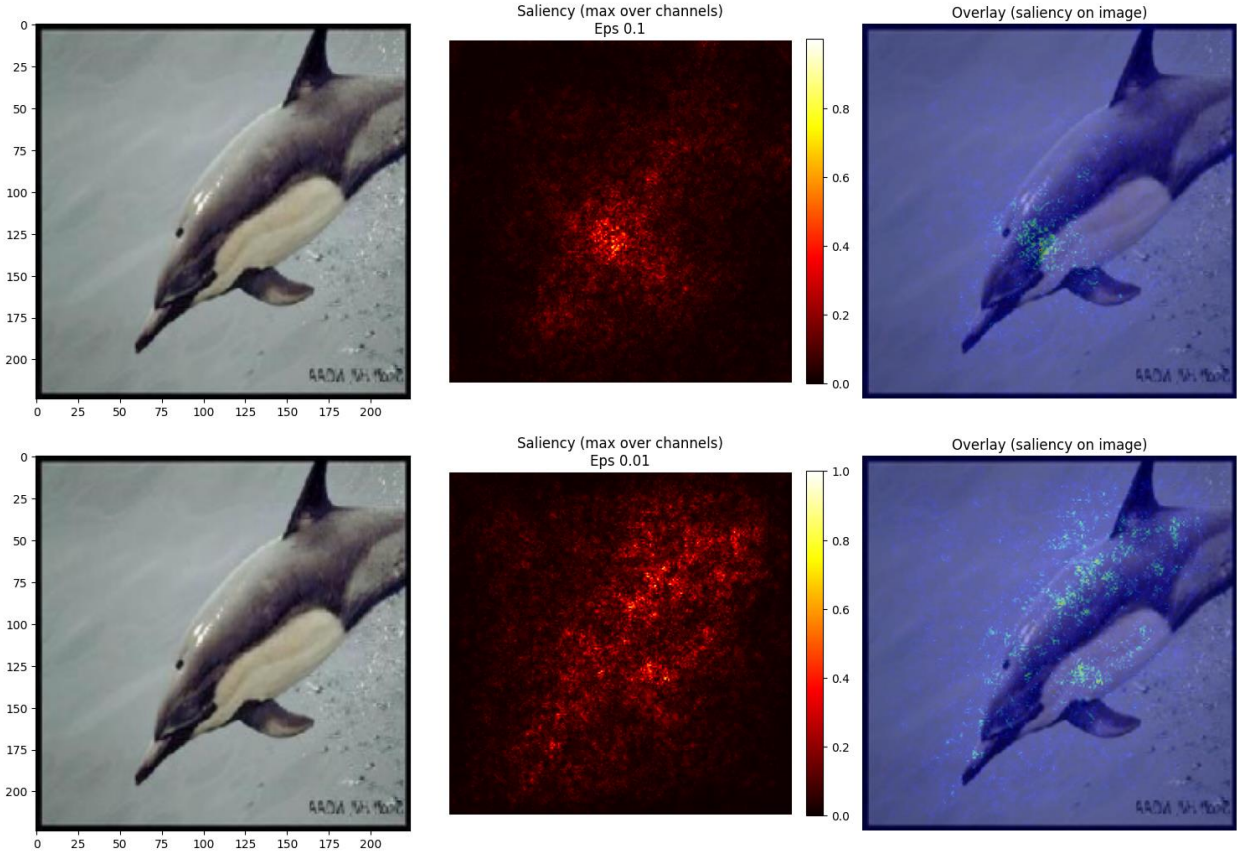

Saliency (max over channels)
Eps 0
0.8
0.6
0.4
0.2
0.0

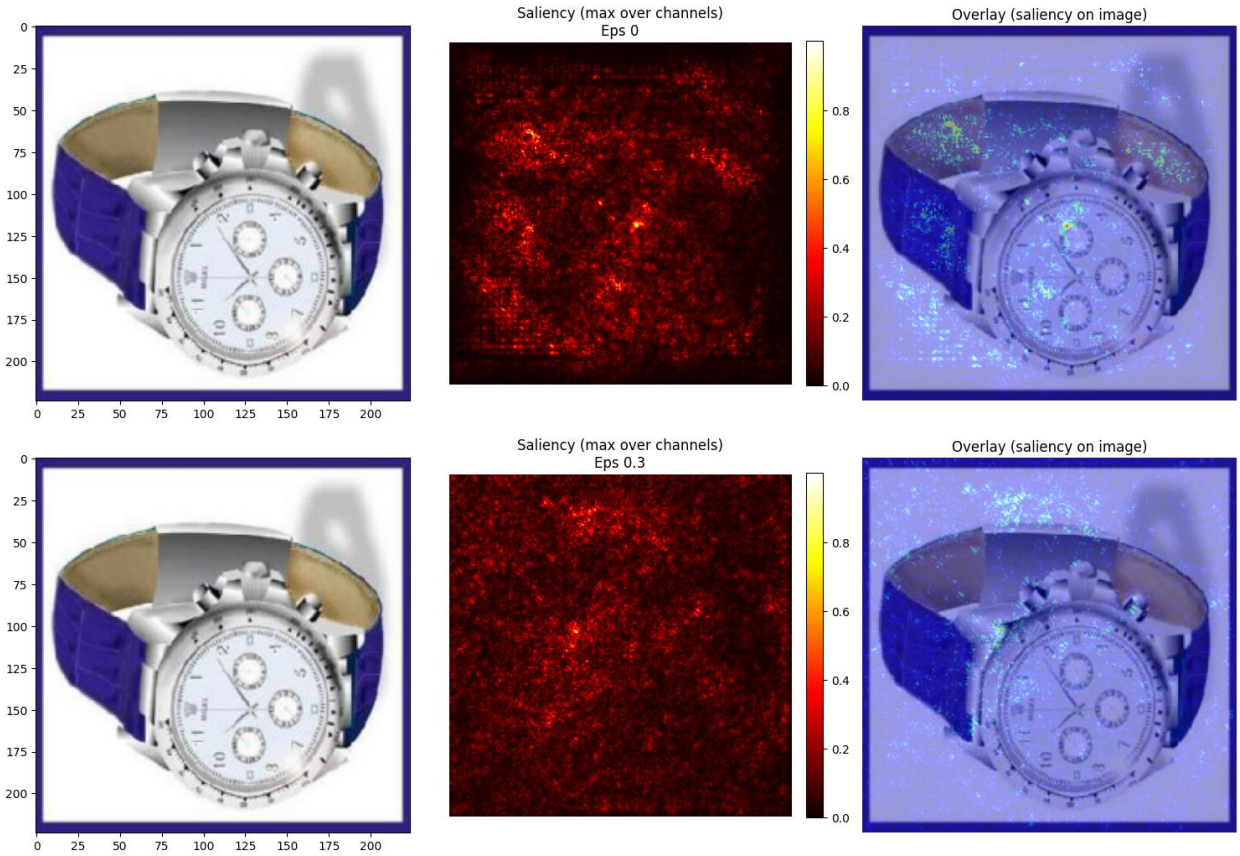
Overlay (saliency on image)
SONY


0
25
50
75
100
125
150
175
200
SONY

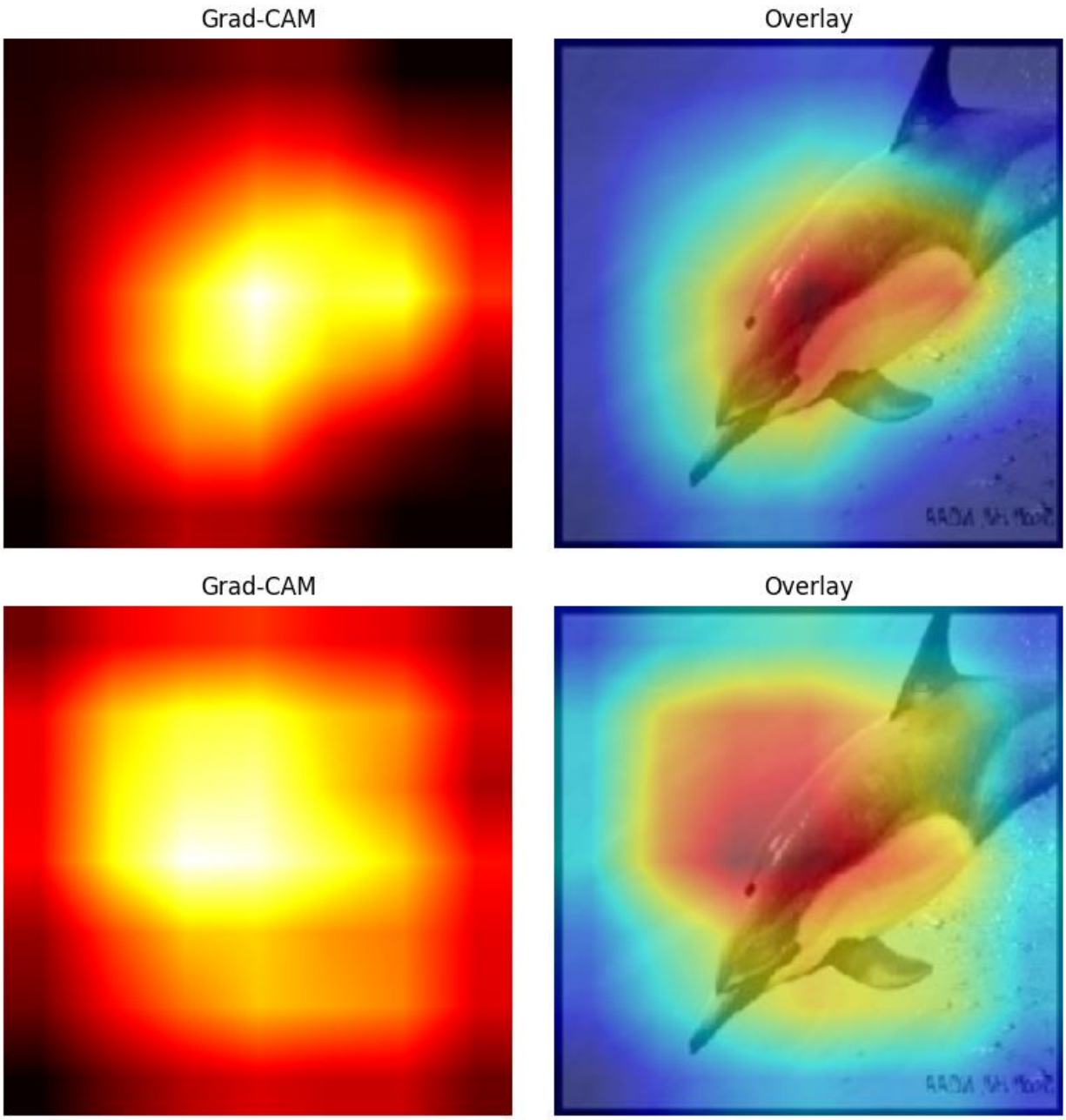

Saliency (max over channels)
Eps 0.2
0.8
0.6
0.4
0.2
0.0

Overlay (saliency on image)
SONY


Saliency (max over channels)
Eps 0.1
Overlay (saliency on image)
Saliency (max over channels)
Eps 0.01
Overlay (saliency on image)

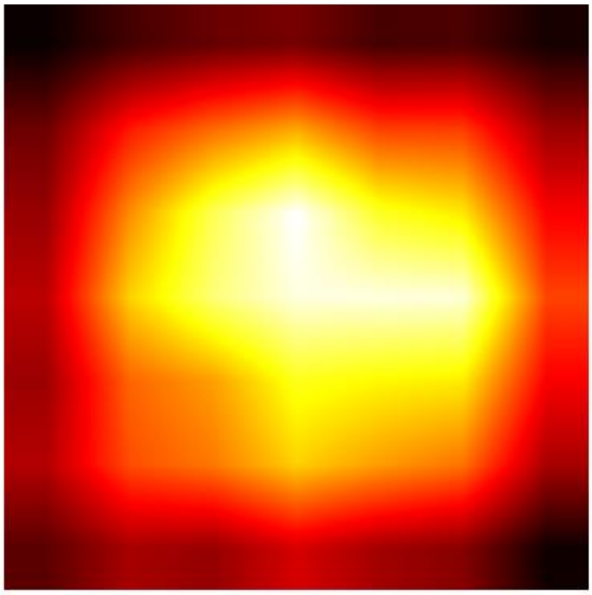


The clear difference between 2 saliency maps shows the most important pixels affecting model classification.
As observed with high epsilon the model focuses on unimportant pixels and non relevant for the target label

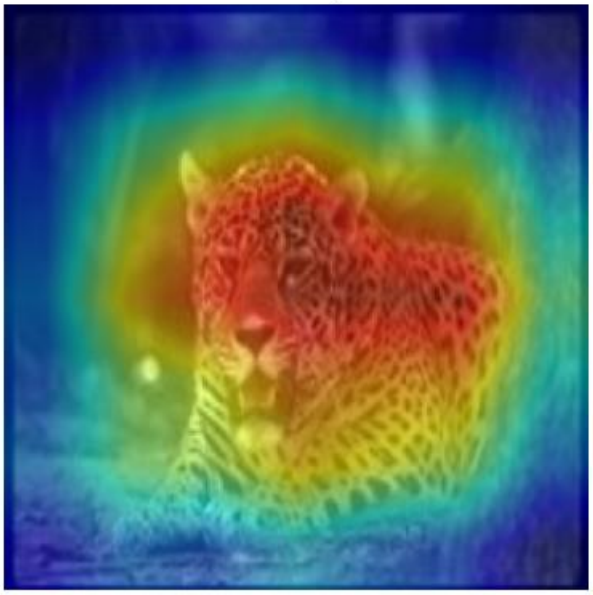


Here GradCam explain the regions rather than the pixels. The first one model focused entirely on the dolphin body while in the second one it shifted its concentration to the background

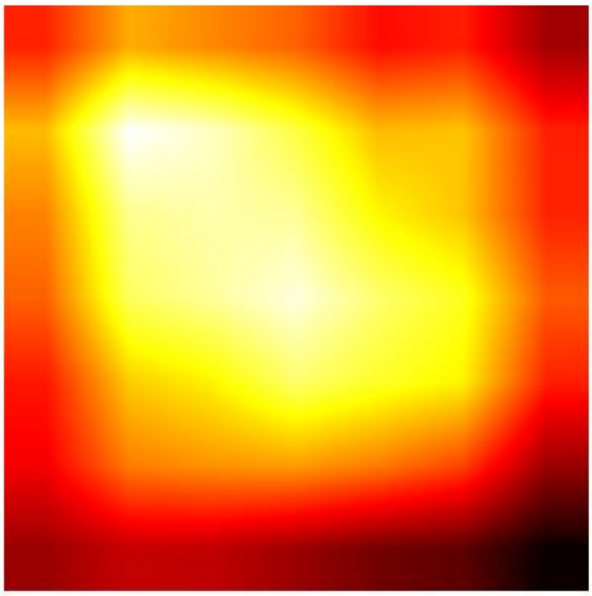
Grad-CAM



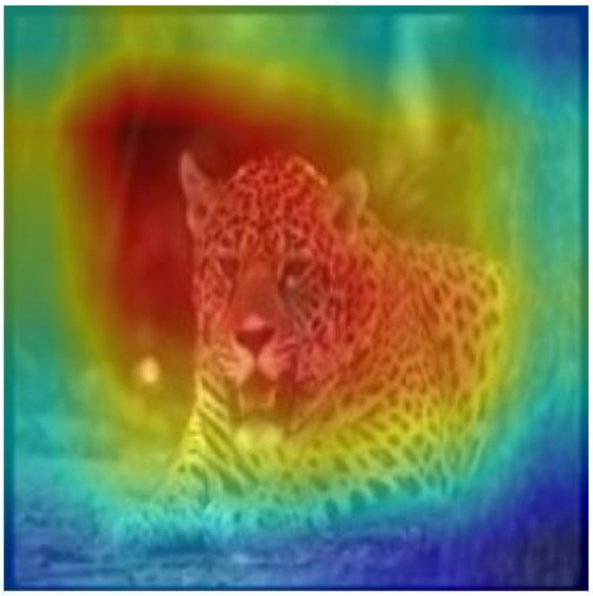
Overlay



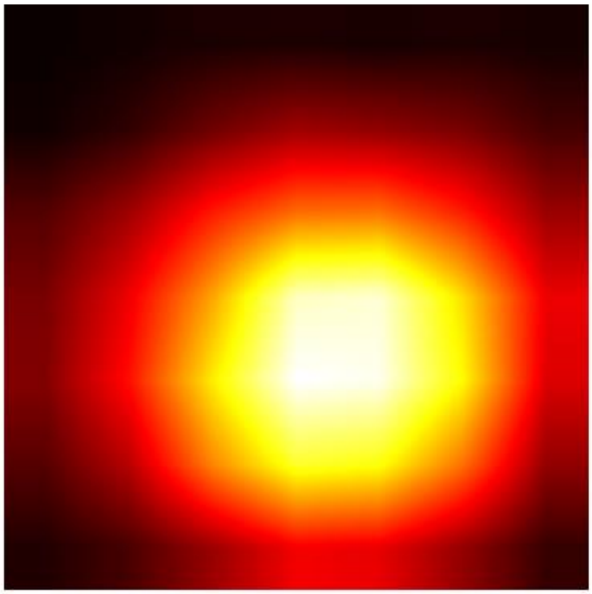
Grad-CAM



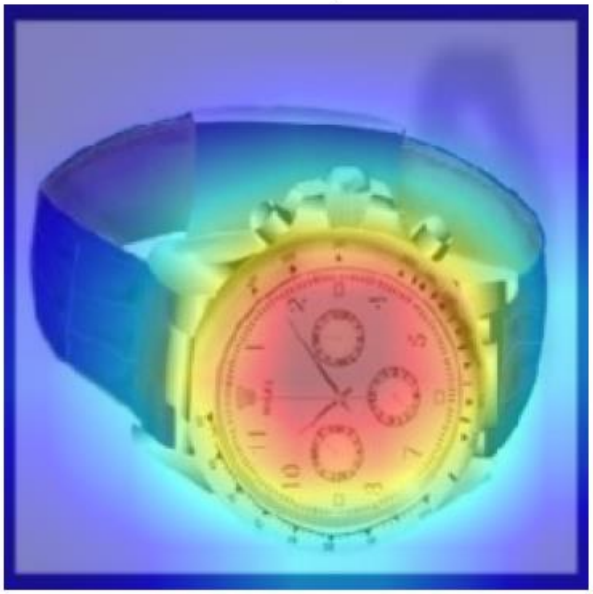
Overlay



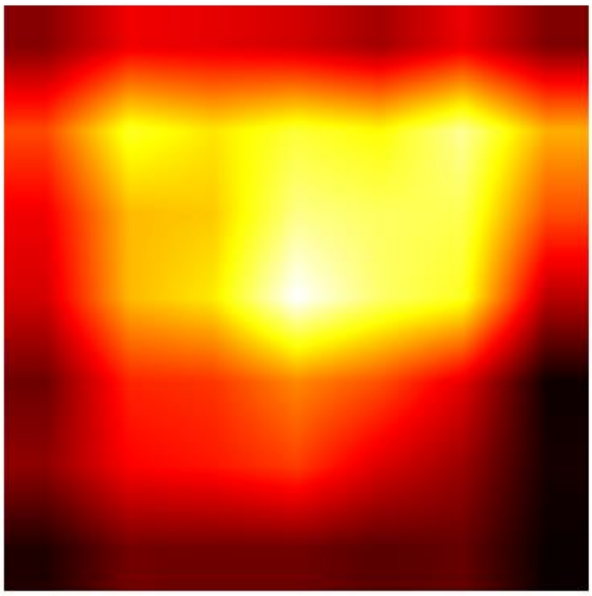
Grad-CAM



Overlay



Grad-CAM



Overlay